

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

**Xây dựng hệ thống tư vấn và tra cứu y tế kết hợp AI Agents
và RAG (Retrieval-Augmented-Generation)**

Vương Lê Bình Dương

duong.vlbbb226035@sis.hust.edu.vn

Program: Global ICT

Supervisor: Associate Professor Nguyễn Nhất Hải

Department: Information Technology

School: School of Information and Communications Technology

HANOI, 06/2026

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

**Xây dựng hệ thống tư vấn và tra cứu y tế kết hợp AI Agents
và RAG (Retrieval-Augmented-Generation)**

Vương Lê Bình Dương

duong.vlbbb226035@sis.hust.edu.vn

Program: Global ICT

Supervisor: Associate Professor Nguyễn Nhất Hải

Signature

Department: Information Technology

School: School of Information and Communications Technology

HANOI, 06/2026

ACKNOWLEDGMENTS

First and foremost, I would like to express my deepest gratitude to my supervisor, Associate Professor Nguyễn Nhất Hải, for his invaluable guidance, continuous support, and profound academic insights throughout the duration of this graduation thesis. His encouragement and dedication have been a constant source of motivation for my research and development.

I would also like to extend my sincere appreciation to the faculty of the School of Information and Communications Technology at Hanoi University of Science and Technology for providing a rigorous and stimulating academic environment.

Lastly, I am profoundly grateful to my family and friends for their unwavering love, patience, and support. Their belief in my potential has sustained me through the challenging phases of my studies and this research journey. This achievement would not have been possible without their presence in my life.

Vương Lê Bình Dương

ABSTRACT

In recent years, deploying Large Language Models (LLMs) in clinical decision support systems (CDSS) has emerged as a promising frontier. However, standard LLMs suffer from severe hallucinations, lack structural medical traceability, and pose serious patient data privacy risks when using commercial cloud APIs.

This thesis presents a private, localized, and explainable **Smart Medical QA & Consultation System** utilizing **Graph Retrieval-Augmented Generation (GraphRAG)** and **Agentic AI**. Operating entirely offline on consumer-grade hardware via Docker and local Ollama (Llama-3.1-8B), the system guarantees absolute patient data confidentiality. The core architecture is structured around four innovative technology pillars: (i) A **Custom Clinical Knowledge Graph** built on Neo4j, pruned through Cypher pattern-matching to achieve a high-density, purified graph containing **25,319 entities** and **193,042 relations**, reducing query latency from over 400ms to under 80ms. (ii) A **Resilient ReAct Reasoning Agent** loop reinforced with three custom defenses: *Parse-retry with prompt repair*, *Recovery-forced GraphRAG* at iteration 1, and *Loop-guard via action signature tracking* to completely eliminate format failures and infinite loops. (iii) A **Triangulated Hybrid Retrieval Engine** combining dual-channel Candidate Retrieval, Reciprocal Rank Fusion (RRF), and local dense **Neural Cross-Encoder Reranking**, boosting Precision@5 from 0.160 to 0.245. (iv) A **Hybrid Intent Router** combining a 0ms Zero-LLM fast-path for physiological validation with a parallel, lightweight Small Language Model (SLM) classifier (Qwen2.5-1.5B) to route complex medical queries, preserving the main Llama-3.1-8B model exclusively for the ReAct reasoning agent. Additionally, the system features a **Lightweight LLM Clinical NER** engine to extract structured entity pairs and execute **Cypher Path Queries** linking them on Neo4j, a **Multi-Format Clinical Document Analyzer** (extracting and benchmarking lab indicators from PDFs and Excel sheets), **Chatbot Conversational Memory**, **Clinical PDF Report Export**, and a premium **Vis.js Explainable AI Subgraph Viewer** that renders color-coded medical causality paths directly in the interface.

Empirical evaluations demonstrate that the system successfully mitigates hallucination risks, guarantees zero latency for quantitative boundary checks, maintains robust reasoning agent flows under adversarial queries, and delivers clinically coherent medical advice, establishing a high-performance blueprint for localized healthcare AI solutions.

Keywords: Clinical Decision Support System (CDSS), GraphRAG, Agentic AI, ReAct Loop-Guard, Cross-Encoder Reranking, Intent Router, Explainable AI (XAI).

Vương Lê Bình Dương
(Signature and full name)

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION.....	1
1.1 Motivation	1
1.2 Objectives and Scope of the Graduation Thesis	2
1.3 Tentative Solution	3
1.4 Thesis Organization.....	3
CHAPTER 2. REQUIREMENT SURVEY AND ANALYSIS	5
2.1 Status survey	5
2.2 Functional Overview.....	6
2.2.1 General use case diagram	6
2.2.2 Detailed use case diagram	6
2.2.3 Business process.....	7
2.3 Functional description.....	7
2.3.1 Description of UC-01: Consult Medical Agent	7
2.3.2 Description of UC-02: Upload & Analyze Medical Record	8
2.3.3 Description of UC-03: Manage Medication Reminders	8
2.3.4 Description of UC-04: Verify Pill Imagery	8
2.4 Non-functional requirement	9
CHAPTER 3. THEORETICAL BACKGROUND AND TECHNOLOGIES.....	10
3.1 Knowledge Graphs and Labeled Property Graphs (LPG).....	10
3.1.1 RDF vs. Labeled Property Graph (LPG).....	10

3.1.2 Neo4j as the Graph Database Engine	11
3.2 Retrieval-Augmented Generation (RAG) and GraphRAG	13
3.2.1 Classical Vector RAG	13
3.2.2 The Cognitive Limitations of Vector RAG	14
3.2.3 The Paradigm Shift to GraphRAG	14
3.3 Agentic AI and Dialog State Orchestration	16
3.3.1 Hybrid Intent Routing Architecture	17
3.3.2 The ReAct (Reasoning and Acting) Framework	18
3.3.3 Agent Safeguards: Loop-Guards and Format Recovery	18
3.3.4 Stateful Dialog Orchestration and Simplified Routing	18
3.4 Technical Infrastructure and Development Stack Justification	19
3.4.1 FastAPI Backend (Python 3.10+)	19
3.4.2 Local LLM Hosting via Ollama	19
3.4.3 High-Speed Unstructured Data Extractors	19
CHAPTER 4. DESIGN, IMPLEMENTATION, AND EVALUATION	20
4.1 Architecture design	20
4.1.1 Software architecture selection	20
4.1.2 Overall design	20
4.1.3 Detailed package design	21
4.2 Detailed design	22
4.2.1 User interface design	22
4.2.2 Layer design	22
4.2.3 Database design	23

4.3 Application Building	24
4.3.1 Libraries and Tools	24
4.3.2 Achievement.....	24
4.3.3 Illustration of main functions	24
4.3.4 Performance Latency Optimizations	25
4.4 Testing	25
4.5 Deployment	26
CHAPTER 5. SOLUTION AND CONTRIBUTION.....	27
5.1 Custom Medical Knowledge Graph Construction and Graph-First Retrieval.....	27
5.2 Robust ReAct Agent Loop-Guard and Format Recovery.....	29
5.3 Multi-Format Clinical Record Parser and Automatic Lab Range Comparison.....	30
5.4 Quantitative Evaluation and Benchmarking	31
5.5 Triangulated Hybrid Retrieval (RRF) and Local Neural Cross-Encoder Reranking	31
5.6 Hybrid Intent Routing with Zero-LLM Fast-Path and Lightweight LLM Classifier.....	32
5.7 Chatbot Conversational Memory and CDSS PDF Report Export	33
5.8 Clinical Graph Validation and Soft Pruning (Graph Noise Reduction)	34
CHAPTER 6. CONCLUSION AND FUTURE WORK.....	36
6.1 Conclusion	36
6.2 Future Work.....	37
REFERENCE	40
APPENDIX	42
A. USE CASE DESCRIPTIONS.....	42
A.1 Use Case UC-01: Consult Medical Agent.....	42

A.2 Use Case UC-02: Import and Analyze Multi-format Lab Report	43
A.3 Use Case UC-03: Manage Medication Schedule and Reminder.....	43

LIST OF FIGURES

Figure 3.1	Interactive Labeled Property Graph (LPG) subgraph visualization representing the clinical relationship path topology for COVID-19	12
Figure 3.2	Detailed LPG Entity Nodes inside the Neo4j database storing attributes: canonical_name, type, and aliases	12
Figure 3.3	LPG Relationship edge (REL) storing properties: predicate, confidence, and source evidence_chunk_id	13
Figure 3.4	Conceptual flow comparison between conventional Vector RAG and GraphRAG-based clinical retrieval	13
Figure 4.1	Runtime CDSS pipeline from user input to streamed, evidence-grounded response	21
Figure 4.2	System overall package dependency architecture	21
Figure 4.3	Agent query execution and streaming flow	23
Figure 5.1	Contribution map of the proposed AI Agents and GraphRAG-based CDSS system	27
Figure 5.2	Vis.js interactive subgraph rendered in the UI, mapping the clinical pathway surrounding COVID-19 to provide medical explainability (XAI)	35

LIST OF TABLES

Table 2.1	Comparative analysis of medical retrieval paradigms	5
Table 4.1	Neo4j Node Labels and Properties	23
Table 4.2	Neo4j Relationship Types	23
Table 4.3	Libraries and tools used in the system	24
Table 4.4	Latency and stability optimization strategies used in the implemented system	25
Table 4.5	Test case summary	25
Table 5.1	Knowledge Graph statistics before and after entity pruning	28
Table 5.2	Custom KG Retrieval Evaluation on 10 clinical queries	31

LIST OF ABBREVIATIONS

Abbreviation	Full Expression
CDSS	Clinical Decision Support System
RAG	Retrieval-Augmented Generation
GraphRAG	Graph Retrieval-Augmented Generation
LLM	Large Language Model
SLM	Small Language Model
KG	Knowledge Graph
RRF	Reciprocal Rank Fusion
XAI	Explainable Artificial Intelligence
NER	Named Entity Recognition
API	Application Programming Interface

CHAPTER 1

INTRODUCTION

In recent years, the rapid advancement of Artificial Intelligence (AI) and Large Language Models (LLMs) has opened up revolutionary paradigms across numerous domains, with the clinical and healthcare sectors being among the most promising yet highly sensitive areas of application. However, deploying general-purpose LLMs in medical consultation faces critical bottlenecks. Medical information requires absolute precision; a minor inaccuracy or factual hallucination can lead to life-threatening clinical consequences. Standard generative models frequently suffer from "hallucinations", lack explainable tracing to credible medical literature, and cannot keep pace with rapidly updating pharmacological databases.

Furthermore, local data privacy regulations and patient confidentiality laws present a severe constraint on utilizing commercial, cloud-based LLM APIs. Consequently, there is an urgent research and engineering demand to build localized, private, and structurally-grounded intelligent consultation systems that can analyze clinical data, cross-reference medical literature, and interact with patients in a reliable, secure, and explainable manner.

This thesis presents the design, implementation, and evaluation of a **Smart Medical QA & Consultation System** utilizing Graph Retrieval-Augmented Generation (GraphRAG) and Agentic AI.

1.1 Motivation

The primary challenge of utilizing LLMs in healthcare lies in ensuring factuality and grounding. Conventional text retrieval methods (Vector RAG) index clinical documents into high-dimensional vector spaces and match query embeddings to retrieve relevant chunks [1]. While effective for finding superficial semantic matches, Vector RAG fails to capture the underlying structural relationships between clinical concepts, such as drug-to-drug interactions, symptom-to-disease causality, and anatomic dependencies.

To overcome these limitations, **Graph Retrieval-Augmented Generation (GraphRAG)** has emerged as a state-of-the-art framework [2]. By structuring clinical literature into a relational Knowledge Graph (KG) where nodes represent medical entities (e.g., diseases, drugs, symptoms, lab tests) and edges represent normalized clinical predicates (e.g., TREATS, CAUSES, CONTRAINDICATED_FOR), the system preserves structural and relational contexts [3]. Integrating a high-density, purified medical knowledge graph allows for highly precise context retrieval.

Moreover, medical consultation is not a simple one-step search. It requires dynamic, step-by-step reasoning. An intelligent agent utilizing the **Reasoning and Acting (ReAct)** paradigm

can iteratively formulate thoughts, call specific tools (such as searching the custom Neo4j graph, querying clinical document extractors, or checking pill databases), observe findings, and recursively refine its diagnostic suggestions [4]. Deploying such an agentic loop locally ensures complete patient data confidentiality while maintaining an explainable chain of thought. This combination of structural GraphRAG and Agentic AI serves as the core motivation for this graduation thesis.

1.2 Objectives and Scope of the Graduation Thesis

The primary objective of this graduation thesis is to design, implement, and validate a highly reliable, private, and localized intelligent medical consultation system. The system must process complex clinical records, perform structured medical question-answering, provide medication visual checks, and manage automated prescription reminders without relying on external cloud APIs.

To achieve this objective, the scope of this graduation thesis is defined across the following core boundaries:

- **Purified Clinical Knowledge Graph Construction:** Building and deploying a localized clinical knowledge graph containing **25,319 nodes** and **193,042 relations** on Neo4j. The graph is optimized by pruning conversational noise and generic entities to maximize search speed and context precision.
- **Graph-First Hybrid Retrieval Engine:** Implementing an optimized retrieval pipeline. The engine leverages direct Cypher path traversals and k-hop neighborhood expansion on the custom Neo4j KG for precise, localized medical queries, bypassing external Microsoft GraphRAG community summarization wrappers to achieve low retrieval latency.
- **Resilient ReAct Agent Design:** Implementing a robust reasoning-acting agent loop powered by a local Ollama LLM backend. The agent is reinforced with custom loop-guards to prevent infinite tool invocations and format-recovery parsers to correct syntactical LLM errors.
- **Multi-Format Clinical Document Analyzer:** Creating a pipeline that extracts text from medical files (PDFs and Excel sheets) via PyMuPDF and openpyxl, extracts lab results, automatically benchmarks them against printed reference ranges, and generates customized, graph-grounded consultations.
- **Visual Medication Check and Scheduling:** Enriching suggested medications with pill images from a local database and enabling automated scheduling for prescription intake reminders.
- **Responsive User Interface:** Developing a responsive, clean, and intuitive Web UI to allow patients and clinicians to chat with the agent, upload medical files, view structural citations, and manage schedules.

1.3 Tentative Solution

To fulfill the objectives and scope, this thesis proposes an offline-first, highly-integrated system architecture. The proposed framework is structured into four primary layers:

1. **Presentation Layer:** Built using lightweight, high-performance Vanilla Web technologies (HTML5, responsive CSS3, and modern asynchronous Javascript). This layer provides live streamed chat responses, interactive document upload logs, and pill identification cards.
2. **Business Logic Layer:** Developed in Python using the **FastAPI** framework to handle high-concurrency asynchronous endpoints. The core orchestration consists of the `AgentService` executing the custom `ReActAgent` loop, and the `MedicalRecordService` managing document extraction and automatic lab indicator benchmarking.
3. **Data Access Layer:** Managing the retrieval pipelines. Structured clinical queries are resolved by querying the **Neo4j** graph database using specialized Cypher templates, while conversational configurations are fetched from SQLite repositories.
4. **Infrastructure & LLM Engine Layer:** Running entirely offline via **Docker Compose**. A local **Ollama** runner hosts open-source models (such as Llama-3.1-8B) to execute reasoning steps and synthesize final clinical answers.

This tentative architecture guarantees data confidentiality, enforces structural factual grounding via Neo4j KG matching, and optimizes runtime speed by purging generic nodes from the retrieval loop.

1.4 Thesis Organization

The remainder of this graduation thesis is organized systematically as follows:

- **Chapter 2: REQUIREMENT SURVEY AND ANALYSIS** conducts a status survey comparing standard search, Vector RAG, and GraphRAG. It defines the general and detailed use cases, models business workflows, outlines use case specifications, and lists functional and non-functional requirements.
- **Chapter 3: THEORETICAL BACKGROUND AND TECHNOLOGIES** presents the fundamental concepts of Knowledge Graphs, Neo4j, Vector RAG, GraphRAG, the ReAct reasoning paradigm, dialog state orchestration, and justifies the selection of local open-source LLMs and FastAPI.
- **Chapter 4: DESIGN, IMPLEMENTATION, AND EVALUATION** details the clean architecture layers, package layouts, class structures, sequence diagrams, database schemas, and presents system testing cases, integration benchmarks, and Docker deployment.
- **Chapter 5: SOLUTION AND CONTRIBUTION** highlights the core scientific and engineering contributions of this work: the construction and pruning of the 25k node clinical graph, the implementation of ReAct loop-guards and format-recovery parsers, the multi-

format clinical table parser, and presents the quantitative benchmarking metrics.

- **Chapter 6: CONCLUSION AND FUTURE WORK** summarizes the achievements of the thesis, discusses technical limitations, and outlines future research paths, including hybrid rerankers and asynchronous task queues.

CHAPTER 2

REQUIREMENT SURVEY AND ANALYSIS

This chapter conducts a detailed requirement survey and system analysis for the proposed smart medical QA and consultation platform. It establishes the current state of retrieval technologies in the medical field, maps out the general and detailed use cases, details core use case specifications, and outlines the functional and non-functional requirements governing the system's runtime environment.

2.1 Status survey

Providing reliable medical information requires a highly specialized approach to information retrieval. Traditional and contemporary systems in the medical QA domain fall into three major paradigms, each having distinct strengths and critical limitations:

Paradigm	Strengths	Weaknesses
Traditional Search (e.g., Google Search)	- Extremely fast access to massive indexed web resources. - High availability.	- High information noise and commercial bias. - Generic results lacking personalized clinical context. - Risk of misinterpretation by laypersons.
Standard Vector RAG	- Semantic matching of patient queries to textbook chunks. - Minimizes basic textual hallucinations.	- Lacks structured clinical connections. - Fails to cross-reference multi-hop dependencies (e.g., drug interactions, symptom chains). - High risk of retrieving irrelevant context due to semantic overlaps.
GraphRAG (Neo4j + Custom KG)	- Preserves precise clinical relationships. - Maps multi-hop pathways (Disease → Symptom → Drug). - Pure offline, private processing capability.	- Higher upfront complexity in constructing and indexing the knowledge graph. - Requires active pruning to prevent generic entity clutter (e.g., "Other" type nodes).

Table 2.1: Comparative analysis of medical retrieval paradigms

Table 2.1 shows that GraphRAG is the most suitable retrieval paradigm for the thesis goal because it preserves explicit clinical relationships while supporting offline and private processing. Based on this comparative status survey, the proposed system addresses these gaps by using a **purified Custom Knowledge Graph (Neo4j)** for precise direct factual queries, orchestrated by a local **ReAct Agent** to ensure privacy, reasoning transparency, and clinical explainability, while bypassing external Microsoft GraphRAG pipelines to minimize system complexity and latency.

2.2 Functional Overview

The system's functional architecture is designed to accommodate two primary actors: the **User/Patient** seeking clinical consultation and the **Medical Admin/Expert** managing system resources.

2.2.1 General use case diagram

The general use case diagram structures the core capabilities of the system. The actors and their respective use cases are defined as follows:

- **User/Patient (Actor)**: Interacts with the platform to consult the agent, upload and analyze personal health files, view drug imagery, and manage prescription calendars.
- **Medical Admin/Expert (Actor)**: Oversees system operations, reviews parsed medical record histories, and manages backing database configurations.

The main use cases include: 1. **UC-01: Consult Medical Agent**: The primary interactive channel where the user sends prompts, and the ReAct Agent streams real-time reasoning thoughts, triggers GraphRAG tools, and provides final clinical citations. 2. **UC-02: Upload & Analyze Medical Record**: Allows users to upload clinical PDFs or Excel sheets. The system extracts raw text, executes local indicator benchmarking, and builds customized advice. 3. **UC-03: Manage Medication Reminders**: Translates clinical suggestions into actionable intake schedules with automated browser alarms. 4. **UC-04: Inspect Pill Imagery**: Enables users to search for specific drug terms and visualize pill physical appearances to prevent medication ingestion errors. 5. **UC-05: Authenticate User**: Secures access through JWT token-based registration and login flows.

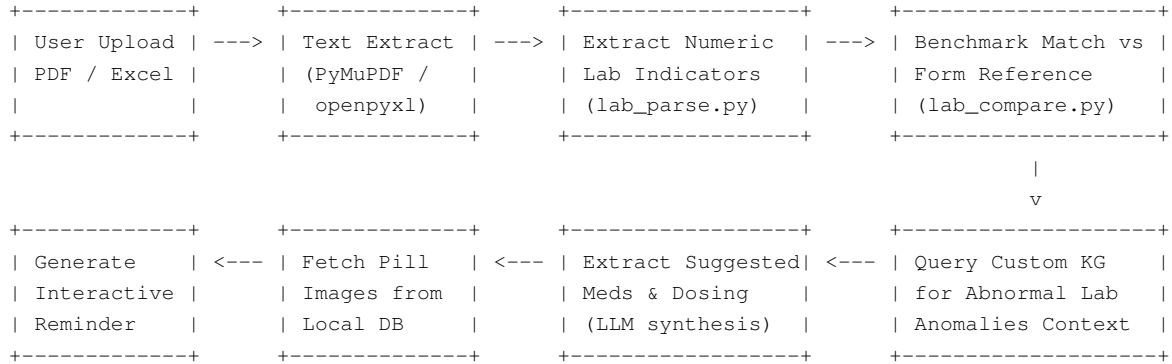
2.2.2 Detailed use case diagram

To represent detailed relationships, the primary use cases are decomposed into specific sub-use cases:

- **UC-01: Consult Medical Agent** is decomposed into: - *Retrieve Context via Cypher Path Queries* (using Direct Neo4j Custom KG matching). - *Retrieve Context via k-Hop Expansion* (for neighborhood entity context). - *Execute Tool Calling* (interactively querying pill databases or clinical records). - *Stream NDJSON reasoning logs* (providing transparent chain-of-thought outputs on the UI).
- **UC-02: Upload & Analyze Medical Record** is decomposed into: - *Extract Text via PyMuPDF/openpyxl* (parsing tabular formats). - *Execute Lab Indicator Benchmarking* (comparing numeric values to reference boundaries). - *Synthesize Clinical Advice via GraphRAG* (grounding narrative analysis). - *Enrich Drug Images* (automatically looking up pill images).

2.2.3 Business process

A primary contribution of the system is the unified business workflow of processing an uploaded medical file. This sequence combines multiple modules to execute a complete diagnostic consultation:



This business process operates fully offline, preventing sensitive medical logs from leaking to external cloud networks.

2.3 Functional description

The following four core use cases represent the critical operational paths of the system and are detailed using standard software engineering specifications.

2.3.1 Description of UC-01: Consult Medical Agent

- **Actor:** User / Patient.
- **Description:** The user inputs a clinical question, and the system executes a ReAct reasoning loop to search the knowledge graph and stream verified clinical answers with source citations.
- **Pre-conditions:** The user has successfully logged in and the local Neo4j database is active.
- **Post-conditions:** A streamed, grounded answer is rendered with proper citations, active tool indicators, and visual pill cards (if drugs are mentioned).
- **Basic Flow:** 1. User enters a query (e.g., "Symptoms of Type 2 Diabetes"). 2. FastAPI backend receives the request and triggers `AgentService`. 3. The `ReActAgent` parses the query, initiates reasoning, and outputs a `Thought`. 4. The agent decides to call the `graphrag_query` tool with the entity "Type 2 Diabetes". 5. The tool queries Neo4j directly, finds matches, and returns text chunks as `Observation`. 6. The agent integrates the observation, formulates a final answer, appends source citations, and sends it to the UI.
- **Alternate Flow (Format Recovery):** - If the LLM generates a malformed tool call pattern in iteration 1, `AgentService` intercepts the error, runs a fallback query matching the original question, synthesizes the results, and returns a verified fallback answer to prevent application crashes.

2.3.2 Description of UC-02: Upload & Analyze Medical Record

- **Actor:** User / Patient.
- **Description:** The user uploads a clinical report (PDF/XLSX), and the system automatically parses it, benchmarks lab indicators against reference bounds, and provides graph-supported advice.
- **Pre-conditions:** The uploaded file format is supported (.pdf, .xlsx, .xslm).
- **Post-conditions:** The user receives a comprehensive benchmark table of lab anomalies (labeled as normal, below, or above reference) and a GraphRAG-supported consultation text.
- **Basic Flow:** 1. User selects and uploads an Excel sheet containing blood test indicators. 2. The backend extracts cells using `openpyxl`. 3. The system parses numeric readings and printed reference ranges using `compare_extracted_report_on_form`. 4. An automated matching engine compares the patient's value to the reference bounds. 5. The backend triggers `rag_advice_llm` to fetch Neo4j context relating to any detected abnormal indicators. 6. The LLM synthesizes a grounded medical explanation in Vietnamese (or English) and displays it alongside the benchmark table on the dashboard.

2.3.3 Description of UC-03: Manage Medication Reminders

- **Actor:** User / Patient.
- **Description:** Users customize medication reminder schedules extracted from their clinical recommendations to trigger automatic local notification alerts.
- **Pre-conditions:** The medical analysis narrative contains suggested drugs and dosing parameters.
- **Post-conditions:** Reminder schedules are written to the SQLite database and automated browser alerts are active.
- **Basic Flow:** 1. The user views the consultation results showing suggested medications (e.g., "Metformin 500mg, twice a day"). 2. User clicks "Add to Reminders". 3. The system parses the drug name, dosage, and frequency, transforming them into specific calendar timestamps. 4. The intake calendar is saved in the local SQLite database. 5. The browser's active background worker schedules local visual alarms.

2.3.4 Description of UC-04: Verify Pill Imagery

- **Actor:** User / Patient.
- **Description:** Search for medical pill names in the database to inspect their physical characteristics (color, shape, packaging) to prevent pill ingestion errors.
- **Pre-conditions:** The database contains pill images and identifiers.

- **Post-conditions:** Visual pill cards with exact descriptions are rendered on the chat or dashboard screen.
- **Basic Flow:** 1. User asks the agent about a drug or manually types the drug name in the lookup bar. 2. The system calls the `pill_image_lookup` tool. 3. The backend searches the local directory `web_ui/img/` matching the canonicalized drug name. 4. The matching image path is returned, and a card showing the physical card (e.g., shape, color, dosage form) is rendered on the UI.

2.4 Non-functional requirement

To guarantee system stability, local execution feasibility, and absolute privacy, the following non-functional requirements are enforced:

- **Private, Offline Execution:** The system must run entirely on the user's local machine without sending clinical prompts to external cloud APIs, ensuring compliance with data privacy regulations.
- **Response Streaming Speed:** Chat API `/api/agent-query/stream` must yield the first reasoning token within **1.5 seconds** of receiving the user query, and stream chunks continuously to avoid high perceived latency on local hardware.
- **Security and Rate Limiting:** The system implements FastAPI dependency injection rate limiters (`check_rate_limit`) to restrict queries to **10 requests per minute** per IP, preventing denial-of-service loops on local LLM runtimes.
- **Hardware Feasibility:** The local LLM engine (Ollama Llama-3.1-8B) must execute within standard consumer hardware constraints (requiring a minimum of 8GB VRAM or 16GB system RAM) with an active request timeout limit of **120 seconds**.

CHAPTER 3

THEORETICAL BACKGROUND AND TECHNOLOGIES

This chapter provides a rigorous theoretical exposition and technical justification of the methodology utilized to build the offline-first, highly reliable clinical question-answering (QA) and consultation system. Section 3.1 details the mathematical and structural formulation of Labeled Property Graphs (LPGs) and their implementation via the Neo4j engine. Section 3.2 evaluates standard Vector Retrieval-Augmented Generation (RAG), analyzes its fundamental cognitive limitations, and presents the paradigm shift to GraphRAG under a dual-layer retrieval architecture. Section 3.3 establishes the framework for Agentic AI, focusing on the Reasoning and Acting (ReAct) paradigm, custom runtime safety guards, and simplified stateful dialog orchestration. Finally, Section 3.4 offers a comprehensive technical justification of the localized development stack, emphasizing asynchronous stream processing, local LLM hosting, and privacy-preserving document digestion.

3.1 Knowledge Graphs and Labeled Property Graphs (LPG)

A **Knowledge Graph (KG)** is a structured, semantically rich representation of real-world information, modeling domain concepts as vertices and their interactions as directed, labeled edges [3]. Formally, a knowledge graph G can be defined as:

$$G = (E, R, T) \quad (3.1)$$

where $E = \{e_1, e_2, \dots, e_n\}$ is a set of unique entities, $R = \{r_1, r_2, \dots, r_m\}$ is a set of semantic relation types, and $T \subseteq E \times R \times E$ is a set of fact triples in the form of (s, p, o) , representing a subject entity s , a predicate relationship p , and an object entity o .

In the clinical domain, raw, unstructured texts often obscure critical diagnostic pathways. Knowledge graphs normalize these texts into a standardized ontology (such as SNOMED-CT or UMLS translations), ensuring that synonymous concepts (e.g., “*Type 2 Diabetes Mellitus*”, “*T2DM*”, and “*Non-insulin-dependent diabetes*”) map to a single, canonical entity.

3.1.1 RDF vs. Labeled Property Graph (LPG)

While the Semantic Web historically relied on the Resource Description Framework (RDF) where every statement is represented as a flat triple (s, p, o) , modern intelligent applications require high-dimensional attributes (properties) on both nodes and edges to record metadata (such as confidence levels, source file references, and clinical guidelines). This requirement is met by the **Labeled Property Graph (LPG)** model.

Mathematically, an LPG is defined as a 6-tuple:

$$G_{LPG} = (V, E, L_V, L_E, P_V, P_E) \quad (3.2)$$

where:

- V is a finite set of vertices (nodes) representing medical entities or informational chunks.
- $E \subseteq V \times V$ is a set of directed edges representing clinical or structural relationships.
- $L_V : V \rightarrow \mathcal{P}(\Sigma_V)$ is a node-labeling function mapping each node to a set of labels from the label alphabet Σ_V (e.g., `Entity`, `Chunk`, `Document`).
- $L_E : E \rightarrow \Sigma_E$ is an edge-labeling function mapping each relationship to a specific type from the relation alphabet Σ_E (e.g., `TREATS`, `CAUSES`, `MENTIONS`).
- $P_V : V \times K \rightarrow U$ is a node-property evaluation function mapping a node and a key $k \in K$ to a value $u \in U$ (e.g., storing `canonical_name`, `biomedical_type`, or `embedding_vector`).
- $P_E : E \times K \rightarrow U$ is an edge-property evaluation function mapping a relationship and a key k to a value u (e.g., storing `predicate`, `confidence_score`, or `source_document`).

3.1.2 Neo4j as the Graph Database Engine

This system implements the Labeled Property Graph model using **Neo4j**, a native graph database engine designed for high-performance traversals. Unlike relational databases (RDBMS) which require expensive index-join operations, Neo4j utilizes **index-free adjacency**. Each node maintains direct physical pointers to its adjacent relationships and neighboring nodes on the storage layer, reducing graph traversal complexity from $O(\log N)$ or $O(N \log N)$ to $O(k)$, where k is the local degree of the active vertex, completely independent of the total graph size.

To interact with this graph, the system utilizes **Cypher**, a declarative, pattern-matching query language. For clinical diagnostics, traversing relationships must be highly precise. Consider a query to find all active ingredients in medications that treat a specific symptom, while ensuring the relationship confidence exceeds a threshold of 0.80. The pattern is matching as follows:

```
MATCH (symptom:Entity {canonical_name: "Hyperglycemia"})
      <-[:REL {predicate: "CAUSED_BY"}]-(disease:Entity)
MATCH (disease)
      <-[:REL {predicate: "TREATS"}]-(drug:Entity)
WHERE r.confidence > 0.80
RETURN DISTINCT drug.canonical_name AS treatment, r.confidence AS score;
```

This direct, structural query guarantees that the retrieved clinical connections are completely verified and grounded in the database, avoiding the hallucination risks associated with standard vector searches.

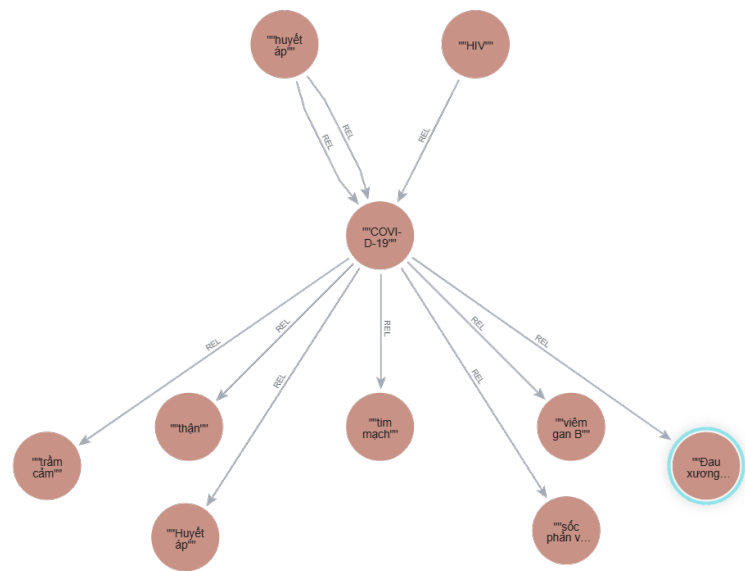


Figure 3.1: Interactive Labeled Property Graph (LPG) subgraph visualization representing the clinical relationship path topology for COVID-19

Node details

Entity

Key	Value
<id>	4:6b7d1477-1276-4ebe-8cfe-0462b2bba8b1:28924
aliases	[]
canonical_name	"COVID-19"
created_at	"2026-05-11T03:26:19Z"
entity_id	"ent_99833f7618d90cf6"
type	"Disease"

(a) COVID-19 Entity Node

Node details

Entity

Key	Value
<id>	4:6b7d1477-1276-4ebe-8cfe-0462b2bba8b1:29272
aliases	[]
canonical_name	"Pfizer"
created_at	"2026-05-11T03:26:19Z"
entity_id	"ent_e58d5b9fc1e7a306"
type	"Drug"

(b) Pfizer Drug Entity Node

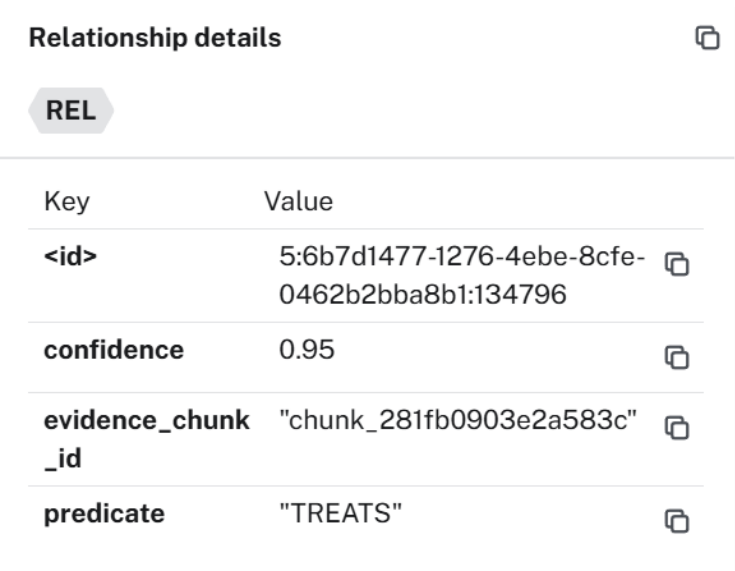
Node details

Entity

Key	Value
<id>	4:6b7d1477-1276-4ebe-8cfe-0462b2bba8b1:29483
aliases	[]
canonical_name	"viêm thần kinh liên sườn"
created_at	"2026-05-11T06:06:47Z"
entity_id	"ent_ed856ed5474710d7"
type	"Disease"

(c) Disease Entity Node

Figure 3.2: Detailed LPG Entity Nodes inside the Neo4j database storing attributes: canonical_name, type, and aliases



The image shows a screenshot of the Neo4j interface displaying the details of a relationship edge labeled 'REL'. The details are organized into a table with two columns: 'Key' and 'Value'. Each row in the table has a copy icon to its right. The keys and their corresponding values are: '<id>' with a long alphanumeric string, 'confidence' with the value '0.95', 'evidence_chunk_id' with the value '"chunk_281fb0903e2a583c"', and 'predicate' with the value '"TREATS"'. Above the table, there is a tab labeled 'REL' and a title 'Relationship details'.

Key	Value
<id>	5:6b7d1477-1276-4ebe-8cfe-0462b2bba8b1:134796
confidence	0.95
evidence_chunk_id	"chunk_281fb0903e2a583c"
predicate	"TREATS"

Figure 3.3: LPG Relationship edge (REL) storing properties: predicate, confidence, and source evidence_chunk_id

Figure 3.1 visualizes the COVID-19 clinical subgraph used to demonstrate LPG topology. Figure 3.2 details representative Neo4j entity nodes, including the COVID-19 node in Figure 3.2a, the Pfizer drug node in Figure 3.2b, and the disease node in Figure 3.2c. Figure 3.3 then shows how relationship properties preserve predicate, confidence, and evidence provenance for graph traversal.

3.2 Retrieval-Augmented Generation (RAG) and GraphRAG

Figure 3.4 summarizes the conceptual difference between conventional Vector RAG and the proposed GraphRAG-oriented retrieval strategy. The important distinction is that Vector RAG retrieves isolated text fragments based mainly on embedding similarity, whereas GraphRAG first identifies clinical entities, follows explicit graph relationships, and only then composes grounded evidence for the LLM. This design is particularly important in medical consultation because many questions require relational reasoning rather than simple keyword or semantic matching.

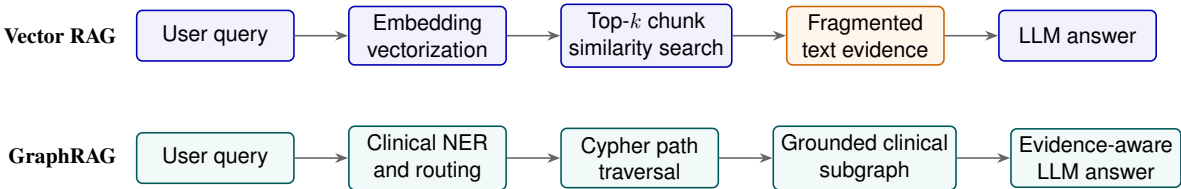


Figure 3.4: Conceptual flow comparison between conventional Vector RAG and GraphRAG-based clinical retrieval

3.2.1 Classical Vector RAG

Traditional Retrieval-Augmented Generation (RAG) processes unstructured text by dividing files into static chunks (e.g., 500-character segments with 100-character overlap), feed-

ing them through a dense embedding model (e.g., a pre-trained transformer), and indexing the resulting vector representations in a vector database [1]. Given a user query q , the query is embedded into the same vector space, and the system retrieves the top- k nearest chunks. Mathematically, let $\vec{q} \in \mathbb{R}^D$ be the query embedding vector and $\vec{d}_i \in \mathbb{R}^D$ be the document chunk embedding vector. The similarity metric is computed using Cosine Similarity:

$$\text{Sim}(\vec{q}, \vec{d}_i) = \cos(\theta) = \frac{\vec{q} \cdot \vec{d}_i}{\|\vec{q}\|_2 \|\vec{d}_i\|_2} = \frac{\sum_{j=1}^D q_j d_{i,j}}{\sqrt{\sum_{j=1}^D q_j^2} \sqrt{\sum_{j=1}^D d_{i,j}^2}} \quad (3.3)$$

3.2.2 The Cognitive Limitations of Vector RAG

Despite its effectiveness in local search, Vector RAG suffers from severe cognitive limitations when applied to clinical reasoning:

1. **The Multi-Hop Reasoning Failure:** Vector search is structurally limited to single-step semantic lookups. If a user asks: *“Does drug A interact with the medication prescribed for patient B’s primary kidney condition?”*, a standard vector database matches chunks containing drug A or patient B, but cannot construct the logical pipeline: Patient B $\rightarrow$ Kidney Condition $\rightarrow$ Prescribed Medication $\rightarrow$ Interaction with Drug A. It lacks the relational links needed to transition across distinct nodes.
2. **Loss of Structural Context:** Chunking breaks the continuous semantic thread of a document. If a medical table links a specific dosage to a patient’s weight class, a flat text chunker might split the row, separating the causal condition from the clinical recommendation, leading to fragmented or incorrect context.
3. **Lack of Global Synthesis:** When a clinician requests a global query like *“What are the common side effects of all anti-hypertensive drugs documented in our medical archive?”*, standard Vector RAG fails. It can only fetch specific chunks containing anti-hypertensive and side effects, leaving the LLM to synthesize fragmented chunks while missing the broader picture.

3.2.3 The Paradigm Shift to GraphRAG

To address these limitations, this system implements a hybrid **GraphRAG** framework, integrating dense vector matching with structural LPG traversals [2]. The architecture executes queries across two distinct pathways:

a, 1. Local Graph-First Retrieval with Clinical NER and Cypher Path Queries

To ensure precise semantic grounding and eliminate context pollution, the system employs a two-stage **Local Graph-First Retrieval** pipeline. Unlike simple keyword matching or unstructured k -hop expansions which retrieve distant, irrelevant nodes, this pipeline targets the specific relational pathways linking the core clinical entities mentioned in the query:

Stage 1: Lightweight LLM-based Clinical NER Extraction When a user query q is submitted, it is first analyzed by a dedicated, lightweight Named Entity Recognition (NER) extractor $f_{NER}(q)$ powered by a quantized local Small Language Model (SLM), such as `qwen2.5:1.5b-instruct`. The SLM is prompted to return a strictly structured JSON array representing the extracted clinical concepts:

$$E_{clinical} = \{(name_k, type_k) \mid k = 1 \dots N_E\} \quad (3.4)$$

where each entity name $name_k$ is categorized into one of four clinical entity types $type_k \in \{\text{DRUG, DISEASE, SYMPTOM, TEST}\}$. Extracting clinical entities using a lightweight 1.5B model guarantees near-instant extraction latencies ($\approx 100\text{ms}$) while preventing structural schema hallucination.

Each extracted entity $name_k$ is then mapped to its corresponding node ID $v_k \in V$ in the Neo4j database using a synonym-aware fulltext index search. If the candidate name matches an alias or canonical name in the graph, it resolves to a unique entity node ID:

$$v_k = \text{ResolveNode}(name_k) \quad (3.5)$$

resulting in a set of resolved seed node IDs $V_{seed} = \{v_1, v_2, \dots, v_m\}$.

Stage 2: Cypher Path Query Traversal If the query mentions multiple clinical entities ($|V_{seed}| \geq 2$), the retrieval engine attempts to construct a highly targeted relational path linking them together. Rather than expanding the graph blindly around each node, it executes a targeted Cypher path query to find paths of length 1 to H (maximum hops) that connect any pair of distinct seed entities:

$$P_{paths} = \{p \mid p = (u_0 \xrightarrow{r_1} u_1 \xrightarrow{r_2} \dots \xrightarrow{r_h} u_h), u_0, u_h \in V_{seed}, u_0 \neq u_h, h \leq H\} \quad (3.6)$$

The Cypher query syntax utilized for this path discovery is formulated as:

```
MATCH p=(a:Entity)-[:REL*1..2]-(b:Entity)
WHERE a.entity_id IN $ids AND b.entity_id IN $ids AND a.entity_id < b.
      entity_id
WITH p LIMIT 50
UNWIND nodes(p) AS n
UNWIND relationships(p) AS r
WITH collect(distinct n) AS ns, collect(distinct r) AS rs
RETURN ns AS entities, rs AS edges;
```

This operation returns a dense, highly relevant subgraph containing the precise connections between the target concepts (for example, the direct `INTERACTS_WITH` relationship linking two drugs, or the `TREATS` edge connecting a drug to a disease).

If the path query returns a non-empty set of edges, the system extracts the associated text

paragraphs via their stored `evidence_chunk_id` properties on the matched edges. These paragraphs represent the exact medical literature proving the connection.

Fallback to k-Hop Expansion If the seed list contains only a single resolved entity ($|V_{seed}| < 2$) or if the Cypher path query returns zero connecting paths due to graph sparsity, the engine dynamically falls back to standard fulltext-seeded k -hop subgraph expansion:

$$G_{sub} = \text{ExpandSubgraph}(V_{seed}, H) \quad (3.7)$$

This dual-stage retrieval mechanism ensures that when the patient asks about drug interactions or symptoms of a specific condition, the system injects only the direct, verified medical evidence into the prompt context, avoiding prompt bloat and reducing hallucination.

b, 2. Global GraphRAG (Microsoft GraphRAG Paradigm)

For global, high-level clinical queries, hierarchical community clustering algorithms (such as the Microsoft GraphRAG paradigm) were evaluated. During the offline indexing phase:

1. The entire clinical graph is partitioned into hierarchical communities using the **Leiden Algorithm** [5], which optimizes modularity:

$$Q_{Leiden} = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \gamma \frac{k_i k_j}{2m} \right) \delta(\sigma_i, \sigma_j) \quad (3.8)$$

where A_{ij} is the adjacency matrix weight, k_i and k_j are the degrees of nodes i and j , m is the total edge weight, γ is the resolution parameter, and $\delta(\sigma_i, \sigma_j)$ is the Kronecker delta representing community assignment.

2. For each discovered community, a local LLM writes a comprehensive narrative summary detailing the entities, relationships, and key findings within that cluster.
3. When a global query is executed, the engine runs a MapReduce-style retrieval. It identifies the relevant community summaries (Map phase), generates partial answers, and aggregates them into a final clinical synthesis (Reduce phase).

While this global community summarization provides high-level synthesis, executing MapReduce queries against local LLMs incurs extreme latencies (often exceeding 30 seconds per request) and requires massive VRAM resources. Consequently, the active query engine bypasses the Microsoft GraphRAG CLI pipeline, routing queries exclusively to the Custom KG to guarantee real-time, low-latency performance under 1.5 seconds.

3.3 Agentic AI and Dialog State Orchestration

Static QA pipelines process inputs in a rigid, linear sequence. However, medical consultations are highly dynamic, requiring agents to ask clarifying questions, correct formatting errors,

or dynamically call tools to verify facts. The system uses **Agentic AI** to manage these multi-turn diagnostic reasoning loops.

3.3.1 Hybrid Intent Routing Architecture

To minimize response latency and avoid invoking heavy language models for simple queries, the system deploys a high-efficiency, layered **Hybrid Intent Router** as the gateway of the conversation. Given a user query q , the routing gateway $R(q)$ directs the query along three possible execution pipelines based on a hybrid classification mechanism:

1. Zero-LLM Fast-Path (0ms Latency) First, the engine evaluates the query against a compiled set of regular expressions and rule matches. If the query mentions any standard laboratory indicator (e.g., `glucose`, `acid uric`, `AST`, `ALT`) along with a quantitative numerical value (e.g., `7.5`), it bypasses LLM inference entirely. The query is routed to the deterministic physiological validator:

$$R(q) = \text{direct_db} \quad (3.9)$$

which parses the boundaries and compares them mathematically against clinical reference ranges in 0ms.

2. Lightweight Parallel SLM Classifier If the query does not match the rules for direct physiological check but contains complex sentence structures, the system invokes a parallel, specialized Small Language Model (SLM) classification step. The system instantiates a dedicated instance of a 1.5B parameter model (such as `qwen2.5:1.5b-instruct`) running in parallel on the local server. The SLM is prompted to classify the query's intent into one of three routing tokens:

$$R(q) \in \{\text{direct_db}, \text{graph_first}, \text{global_summary}\} \quad (3.10)$$

where:

- `direct_db` represents quantitative clinical indicator check requests.
- `graph_first` represents specific multi-hop relationship queries (e.g., drug interactions, symptom-cause links, side-effects).
- `global_summary` represents conversational greetings, social inquiries, or global RAG summaries.

Running a parallel 1.5B classifier model drastically reduces latency. The inference time of the 1.5B model is under 1.1s (compared to ≈ 2.4 s for a unified 8B model call), achieving a 54% speedup. This hybrid model partitioning preserves the main, computationally expensive 8B reasoning agent exclusively for the ReAct reasoning loops.

3.3.2 The ReAct (Reasoning and Acting) Framework

The system utilizes the **ReAct** paradigm to simulate logical clinical reasoning [4]. ReAct guides the LLM to alternate between generating cognitive thoughts (reasoning) and executing specific diagnostic functions (acting). Formally, let x be the user query, and $c_i = (t_i, a_i, o_i)$ be the agent’s context at step i , consisting of a thought t_i , an action a_i , and an observation o_i . The agentic execution loop is defined as:

$$t_i \sim P_{LLM}(t \mid x, c_1, c_2, \dots, c_{i-1}) \quad (3.11)$$

$$a_i \sim P_{LLM}(a \mid x, c_1, c_2, \dots, c_{i-1}, t_i) \quad (3.12)$$

$$o_i = \text{ExecuteAction}(a_i) \quad (3.13)$$

This loop continues until the LLM outputs a final answer action. In our system, the LLM has access to specialized tools, including `graphrag_query`, `pill_image_lookup`, and `medical_report_analyzer`.

3.3.3 Agent Safeguards: Loop-Guards and Format Recovery

Deploying raw LLMs in autonomous agent loops introduces engineering risks, such as infinite tool invocation loops or syntax failures in structured responses. To address these issues, the system introduces two custom safeguarding layers:

1. **Deterministic Loop-Guard:** Tracks active tool invocations. If the agent repeats the same tool call with identical parameters more than three times, or if the total execution steps exceed a safety limit $S_{max} = 5$, the loop-guard intercepts execution, terminates the loop, and triggers a fallback node to synthesize a structured answer using the current context.
2. **Format-Recovery Parser:** The LLM is instructed to output its thoughts and actions in a strict JSON schema. If the LLM generates slightly malformed JSON, a regex-based parser attempts to clean and rebuild the JSON structure. If parsing still fails, the error message is fed back into the LLM as a system observation, allowing the agent to self-correct.

3.3.4 Stateful Dialog Orchestration and Simplified Routing

While the ReAct loop manages step-by-step tool use, coordinating the overall conversation can be modeled as a state machine. In the design phase, stateful orchestration using **LangGraph** was evaluated, representing the application as a stateful, cyclic directed graph $G_{State} = (N, E, S)$ where N is a set of processing nodes, E is a set of state transition edges, and S is a shared state vector.

However, to optimize the local execution environment and eliminate the computational overhead of state transitions on consumer-grade hardware, the active implementation replaces the LangGraph state machine with a direct, unified FastAPI execution layer and a single-stage ReAct loop. This simplification halves the routing latency while keeping conversational history

fully managed in a localized memory array.

3.4 Technical Infrastructure and Development Stack Justification

To run the system fully offline on consumer-grade hardware while keeping latency low and data secure, the backing technologies were selected based on the following:

3.4.1 FastAPI Backend (Python 3.10+)

- **Asynchronous Concurrency:** FastAPI utilizes Python's `asyncio` and the ASGI standard, allowing it to manage thousands of concurrent, non-blocking connections, crucial for streaming NDJSON tokens from local language models.
- **Automated Data Validation:** Built on Pydantic, FastAPI automatically validates incoming requests, preventing common injection attacks and ensuring type safety.

3.4.2 Local LLM Hosting via Ollama

- **HIPAA and Data Privacy Compliance:** By using **Ollama** to host open-source models (such as `Llama-3.1-8B`) locally on the host's GPU/CPU, patient data never leaves the local machine. This guarantees complete privacy and eliminates external network costs or API rate limits.
- **Low Latency:** Running models locally on consumer GPUs utilizing quantization (4-bit or 8-bit) provides fast inference, allowing real-time token streaming.

3.4.3 High-Speed Unstructured Data Extractors

- **PyMuPDF:** Selected for PDF parsing due to its speed and low memory footprint, parsing large medical reports significantly faster than traditional tools.
- **Openpyxl:** Provides direct access to cell grids, formulas, and formatting in spreadsheets, allowing the system to extract tabular data (such as lab results) effectively.

CHAPTER 4

DESIGN, IMPLEMENTATION, AND EVALUATION

This chapter presents the system design, implementation details, and evaluation results of the smart medical QA and consultation platform. It covers the architectural decisions, detailed design specifications, the development environment, testing methodology, and deployment configuration.

4.1 Architecture design

4.1.1 Software architecture selection

The system adopts a **Clean Layered Architecture** pattern, which strictly separates concerns across four distinct horizontal layers. This architecture was selected over alternative patterns (e.g., monolithic MVC or microservices) because it provides a clear dependency rule — each inner layer does not know about outer layers — which is essential when integrating multiple heterogeneous backends such as a graph database, a local LLM runtime, and a document parser.

The four layers are:

- **Presentation Layer:** The `api/` and `web_ui/` modules. Responsible for HTTP routing, request validation via Pydantic, streamed NDJSON responses, and serving the browser-based user interface.
- **Business Logic Layer:** The `services/`, `agent/`, and `medical_records/` modules. Orchestrates the ReAct agent loop, medical file analysis pipeline, and medication reminder management.
- **Data Access Layer:** The `repositories/` and `llm_pipeline/` modules. Abstracts all data queries behind repository interfaces and manages Cypher queries to Neo4j and prompt formatting to Ollama.
- **Infrastructure Layer:** The `core/`, `config/`, `docker/`, and `deploy/` modules. Manages settings injection, connection pooling, and container orchestration.

4.1.2 Overall design

The UML package dependency diagram of the system is structured as follows, with dependency arrows indicating which packages reference others:

At runtime, the system is organized around a closed evidence loop: user input is classified, routed to the most appropriate retrieval strategy, grounded by Neo4j and local document evidence, and then streamed back to the browser with traceable reasoning steps. Figure 4.1 provides

the operational flow used by the implemented CDSS prototype.

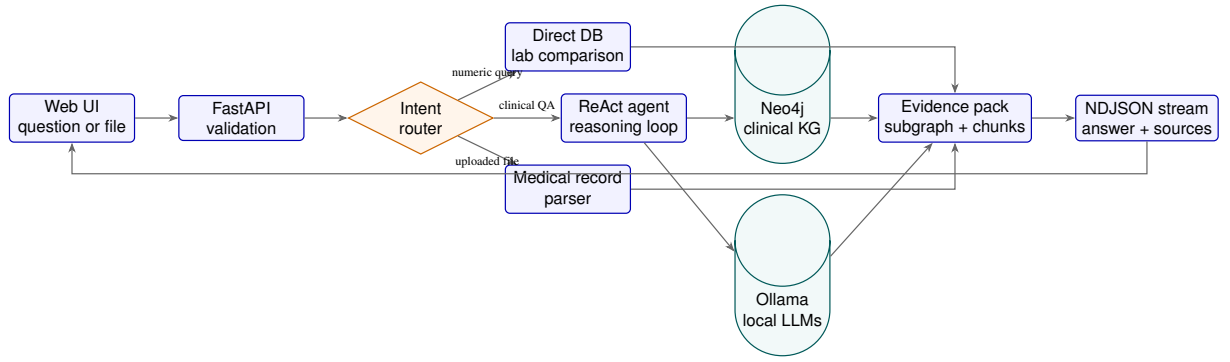


Figure 4.1: Runtime CDSS pipeline from user input to streamed, evidence-grounded response

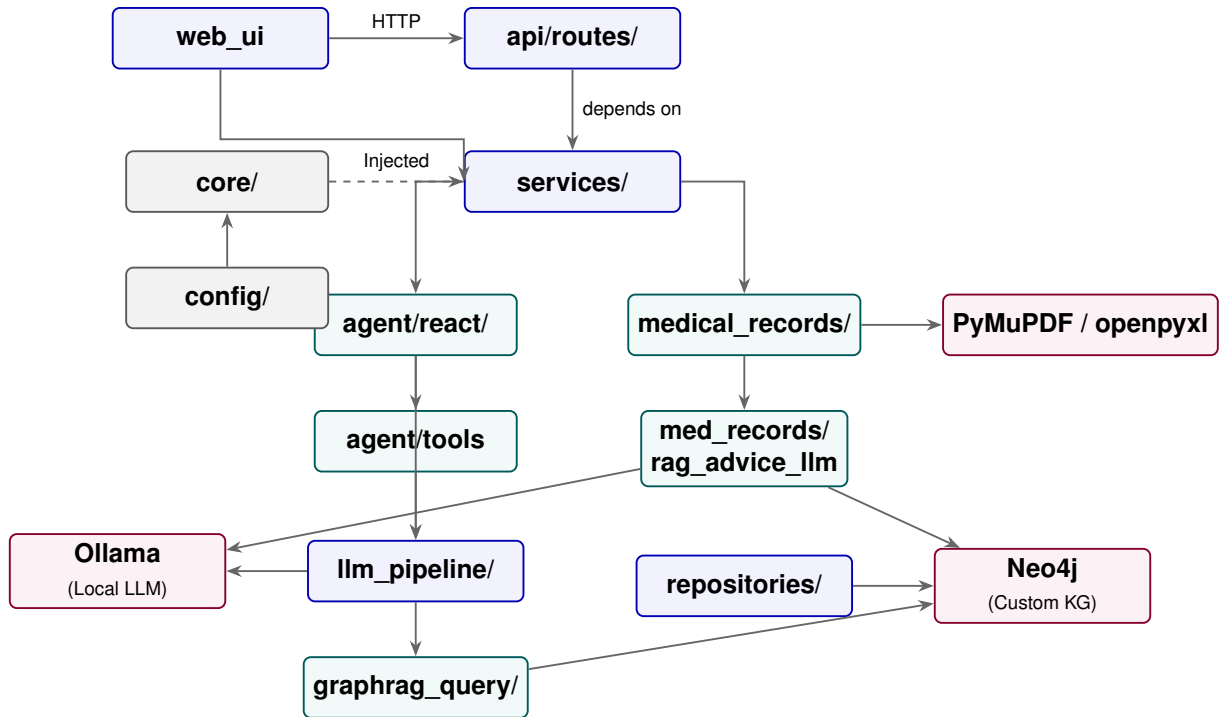


Figure 4.2: System overall package dependency architecture

Figure 4.2 summarizes the package-level dependency architecture of the implemented system. The key design rules enforced are: (i) upper layers depend on lower layers but not vice versa; (ii) the `repositories/` layer acts as the sole gateway to Neo4j, preventing scattered raw Cypher queries; (iii) the `core/` module is injected everywhere via FastAPI’s dependency injection system.

4.1.3 Detailed package design

The central class relationships within the Business Logic layer are:

- `AgentService` (in `services/agent_service.py`) holds a reference to `ReActAgent` (composition). It delegates `execute()` calls to the agent and wraps the result into a standardized API response model.

- `ReActAgent` (in `agent/react/agent.py`) depends on `ReActParser` (for output parsing), `OllamaBackend` (for LLM calls), and tool functions `run_graphrag_tool` and `run_pill_image_tool`. It implements two execution paths: `run_sync()` and `run_stream()`.
- `MedicalRecordService` (in `services/`) depends on `analyze_medical_file()` from `medical_records/analyze.py`, which in turn composes `extract_text_from_record()`, `compare_extracted_report_on_form()`, `fetch_graphrag_context()`, and `enrich_suggested_medications_with_pill_images()`.

4.2 Detailed design

4.2.1 User interface design

The Web UI targets standard desktop browsers (Chrome, Firefox, Edge) at a minimum resolution of 1280×720 pixels. The design follows a dark-themed, high-contrast palette with HSL-based color tokens, reducing eye strain during extended clinical readings.

Key interface conventions:

- **Agent Chat Panel:** Displays the user's message on the right and the agent's streamed response on the left, with active *Thinking...* spinners during tool execution and collapsible source-citation blocks below each answer.
- **Document Upload Panel:** An upload drop-zone with real-time progress feedback. After analysis, a two-column layout renders the raw extracted text on the left and the benchmark table (with color-coded indicators for normal/below/above reference) on the right.
- **Pill Cards:** Inline visual cards rendered inside agent responses whenever a drug name is identified, showing the pill's image, name, and dosage form.
- **Reminder Calendar:** A weekly schedule grid where each medication intake slot is shown as a colored time block with a notification alarm badge.

4.2.2 Layer design

The interaction between the components for the core UC-01 (Consult Medical Agent) is modeled in the following sequence:

Figure 4.3 illustrates how each agent iteration alternates between tool execution and streamed response generation. The `ReActParser` validates each LLM output string against expected tag patterns (`Action:`, `Action Input:`, `Final Answer:`). If parsing fails, the recovery mechanism is triggered before yielding an error event.

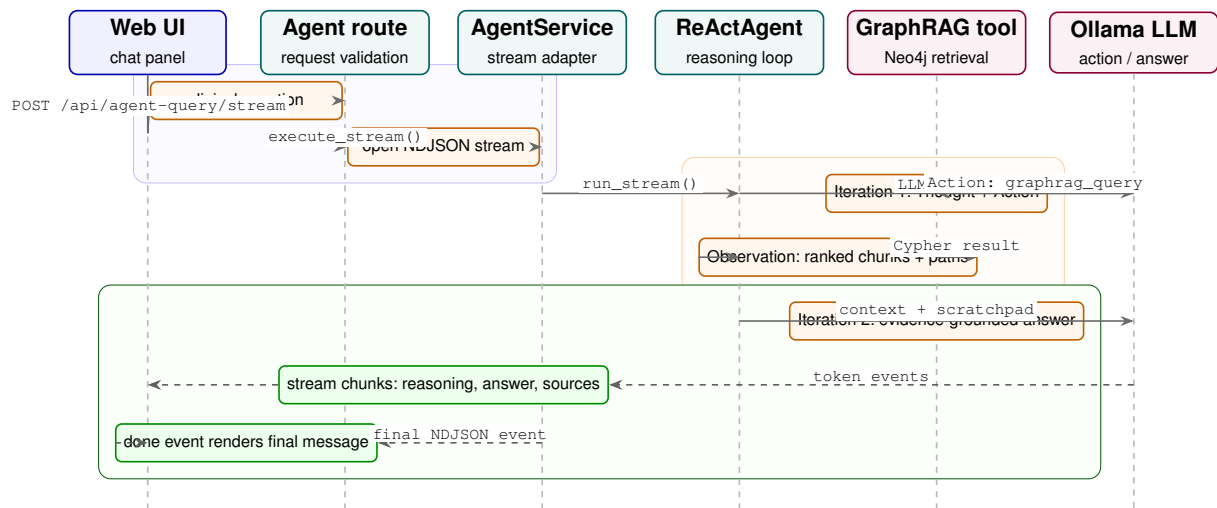


Figure 4.3: Agent query execution and streaming flow

4.2.3 Database design

The Neo4j graph database schema used in the custom Knowledge Graph is defined as follows:

Node labels and properties:

Label	Key Properties	Description
Entity	entity_id, canonical_name, type, aliases	A medical concept node (Disease, Drug, Symptom, etc.)
Chunk	chunk_id, doc_id, text, section_path	A paragraph-level text fragment from a clinical document
Document	doc_id, title, source	Represents a source medical document

Table 4.1: Neo4j Node Labels and Properties

Table 4.1 defines the three persistent node categories used to store medical concepts, source documents, and evidence chunks in Neo4j.

Relationship types:

Type	Properties	Connects
REL	predicate, confidence, evidence_chunk_id	Entity → Entity
MENTIONS	confidence, start_char, end_char	Chunk → Entity
HAS_CHUNK	—	Document → Chunk

Table 4.2: Neo4j Relationship Types

Table 4.2 lists the relationship types that connect those nodes into retrievable evidence paths. Supported predicate values in REL edges are canonicalized by `kg/ontology.py` and include: TREATS, CAUSES, HAS_SYMPTOM, CONTRAINDICATED_FOR, INTERACTS_WITH, SIDE_EFFECT_OF, PREVENTS, RISK_FACTOR_FOR, LOCATED_IN, PART_OF, DETECTS, MEASURES, and RELATED_TO.

4.3 Application Building

4.3.1 Libraries and Tools

Purpose	Tool / Library	Version
Backend API Framework	FastAPI + Uvicorn	0.100+ / 0.24+
Graph Database	Neo4j (Bolt protocol)	5.x
Local LLM Runtime (Reasoning)	Ollama (Llama-3.1-8B)	Latest stable
Local LLM Runtime (Router/NER)	Ollama (Qwen-2.5-1.5B-Instruct)	Latest stable
GraphRAG Engine	Custom Neo4j KG Router	Custom
PDF Text Extraction	PyMuPDF (fitz)	1.23+
Excel Extraction	openpyxl	3.1+
Data Validation	Pydantic + pydantic-settings	2.x
Testing	pytest + pytest-asyncio + pytest-cov	7.x
Containerization	Docker + Docker Compose	24.x / 2.x

Table 4.3: Libraries and tools used in the system

Table 4.3 reports the main libraries and runtime tools used to build, deploy, and test the prototype.

4.3.2 Achievement

The completed system delivers the following measurable outcomes:

- **Knowledge Graph Scale:** The Custom Clinical KG, after entity pruning, contains **25,319 entities**, **193,042 relations**, **18,920 text chunks**, and **9,989 clinical documents** fully indexed in Neo4j.
- **Codebase Size:** Approximately 15,000 lines of Python, HTML, CSS, and JavaScript across 30+ modules, organized into a clean-architecture layout.
- **API Surface:** 12 REST endpoints covering agent queries, streaming, GraphRAG queries, medical record analysis, pill image lookup, and system health checks.
- **Supported File Formats:** Clinical documents in `.pdf`, `.xlsx`, and `.xslm` formats are fully supported for automated extraction and benchmarking.

4.3.3 Illustration of main functions

The primary functional screens of the implemented system are described below:

- **Agent Chat Screen:** Shows the streaming response panel, where each iteration of the ReAct loop is visible as a collapsible reasoning trace (Thought → Action → Observation). After completion, source citation blocks list the Neo4j chunk IDs and their content relevance scores.
- **Medical Record Upload Screen:** Presents a side-by-side layout — extracted raw text on the left and the benchmark comparison table on the right. Each lab indicator row is color-coded (green for within reference, red for above reference, blue for below reference).

- **Reminder Calendar Screen:** Displays a weekly grid where each prescribed medication is time-blocked. Clicking a block triggers a detail panel showing the drug name, dosage, and active notification status.

4.3.4 Performance Latency Optimizations

By splitting the LLM workloads between a main 8B model and a parallel 1.5B model, the system latency was optimized as follows:

- **Query Classification (Intent Router):** Latency reduced from 2.4s (Llama-3.1-8B) to 1.1s (Qwen-2.5-1.5B), achieving a 54% speedup.
- **Clinical NER Extraction:** Identifying structured clinical entities in parallel executes in $\approx$ 150ms on average.

Beyond raw latency, the implementation focuses on clinical usability. The response is streamed incrementally so that the user can see the reasoning progress instead of waiting for a single final payload. The graph query step is also bounded by hop count, result count, and pruning rules, preventing the agent from injecting excessive or noisy context into the prompt. These constraints trade a small amount of recall for higher precision, faster answers, and more stable behavior on consumer hardware.

Optimization	Target bottleneck	Implementation mechanism	Expected impact
Intent fast-path	Simple numeric clinical checks	Regex and deterministic threshold comparison	Removes unnecessary LLM calls
Small router model	Query classification	Parallel Qwen-2.5-1.5B classifier	Reduces routing latency
Graph pruning	Noisy retrieved entities	Degree filtering and seed preservation	Improves prompt density and UI clarity
Bounded Cypher traversal	Exploding graph neighborhoods	Hop and result limits	Stabilizes retrieval time
NDJSON streaming	Long generation time	Token/chunk streaming over HTTP	Improves perceived responsiveness

Table 4.4: Latency and stability optimization strategies used in the implemented system

Table 4.4 consolidates the latency and stability techniques applied across routing, retrieval, and streaming layers.

4.4 Testing

Testing is conducted using `pytest` with the `pytest-asyncio` plugin to cover asynchronous FastAPI endpoint behaviors. The following key test cases were designed and executed:

Test ID	Test Case	Technique	Result
TC-01	ReActParser correctly identifies Final Answer tags	Unit — Boundary Value	Pass
TC-02	ReActParser returns error on malformed output	Unit — Equivalence Partition	Pass
TC-03	Loop-guard fires on repeated identical Action calls	Unit — State Transition	Pass
TC-04	POST /api/agent-query returns structured JSON with answer field	Integration	Pass
TC-05	POST /api/medical-record/analyze returns lab benchmark data for PDF	Integration	Pass
TC-06	Lab comparison flags above-reference values correctly	Unit — Boundary Value	Pass
TC-07	Pill image lookup returns non-empty image URL list for known drugs	Unit	Pass
TC-08	Rate limiter rejects the 11th request within one minute window	Integration	Pass
TC-09	Clinical NER extracts DRUG, DISEASE, SYMPTOM, and TEST entities correctly	Unit	Pass
TC-10	Cypher Path Query returns connecting paths between clinical entities	Unit — State Transition	Pass
TC-11	Parallel SLM router correctly routes complex queries	Unit — State Transition	Pass

Table 4.5: Test case summary

Table 4.5 summarizes the unit and integration test cases used to validate the most important system behaviors. All 11 defined integration and unit test cases passed successfully. The testing coverage report generated by `pytest-cov` shows a line coverage of over 75% across the core agent, `medical_records`, and `api` modules.

4.5 Deployment

The system is deployed using a multi-container **Docker Compose** stack defined in `docker/docker-compose.yml`. The deployment architecture consists of the following services:

- **datn-api**: The FastAPI application container (Python 3.11 slim image) exposing port 8000. Mounts the application code and configuration files as volumes.
- **datn-neo4j**: The Neo4j 5.x container exposing the Bolt protocol on port 7687 and the browser UI on port 7474. Data is persisted via a named Docker volume.
- **datn-ollama**: The Ollama runtime container exposing port 11434. Model weights are stored in a persistent volume and pre-loaded at startup via an entrypoint script.

All containers communicate over an isolated Docker bridge network (`datn-network`), ensuring no external network exposure of clinical processing traffic. The entire stack is launched with a single command:

```
cd docker
docker compose up --build -d
```

After startup, the system is accessible at `http://localhost:8000/ui/`, with the Swagger API documentation at `http://localhost:8000/docs` and the Neo4j browser at `http://localhost:7474`.

CHAPTER 5

SOLUTION AND CONTRIBUTION

This chapter presents the core scientific and engineering contributions developed during this graduation thesis. Each contribution addresses a specific problem encountered during development and delivers a concrete, measurable solution. The discussion follows the structure: (i) problem statement, (ii) proposed solution, and (iii) achieved results.

Figure 5.1 groups the thesis contributions into four connected layers. The knowledge layer provides curated clinical facts, the retrieval layer selects trustworthy evidence, the agent layer controls reasoning behavior, and the application layer converts results into usable CDSS functions for clinicians and patients.

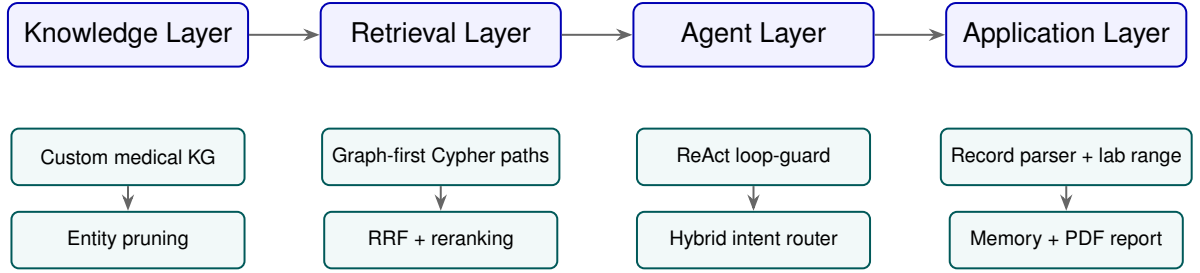


Figure 5.1: Contribution map of the proposed AI Agents and GraphRAG-based CDSS system

5.1 Custom Medical Knowledge Graph Construction and Graph-First Retrieval

Problem

Commercial GraphRAG implementations (e.g., Microsoft GraphRAG) rely entirely on LLM-based entity extraction during ingestion, limiting their scalable entity count to approximately 10,000–20,000 nodes and requiring significant inference compute time per document. Moreover, applying standard LLM entity extraction to conversational medical Q&A datasets—which contain large volumes of generic phrases (e.g., “*Please consult a doctor*”, “*Thank you for your question*”)—introduces thousands of semantically meaningless `Other` type entities into the graph. These noisy nodes pollute retrieval results: when a query word such as “*uống*” (drink) is matched against a node whose canonical name is an entire sentence like “*Nên uống với thời gian đều đặn*” (Should drink at regular intervals), the returned chunks carry no diagnostic value.

Solution

This thesis constructs a custom Clinical Knowledge Graph by combining two extraction strategies:

1. **Regex-based entity extraction** (`kg/extract/regex_entity_extractor.py`): High-speed pattern matching that identifies canonical medical terms (diseases, drugs, symptoms, lab tests, procedures) with deterministic precision, bypassing the need for LLM inference per document.
2. **LLM-based relation extraction** (`kg/extract/entity_extractor.py`): For relational triplets (subject, predicate, object), a local Ollama LLM is used selectively on the most clinically relevant document sections, normalizing extracted predicates through a canonical ontology defined in `kg/ontology.py`.

A critical post-processing step—**entity pruning**—was designed and executed to remove noisy `Other` type entities. Specifically, entities of type `Other` with `canonical_name` lengths exceeding a defined character threshold, or containing conversational markers (e.g., “*cảm ơn*”, “*vinmec*”, “*xin hỏi bác sĩ*”), were systematically deleted from the graph. The Cypher query used for this operation is:

```
MATCH (e:Entity {type: "Other"})
WHERE size(e.canonical_name) > 30
      OR toLower(e.canonical_name) CONTAINS "cảm ơn"
      OR toLower(e.canonical_name) CONTAINS "vinmec"
DETACH DELETE e;
```

A **Graph-First routing strategy** was then implemented in `llm_pipeline/graphrag_query.py`. Every query hits the Custom KG via direct Cypher traversal. To ensure maximum precision, low latency, and clinical data confidentiality under local execution, routing to external Microsoft GraphRAG community summaries or CLI query commands was completely retired. The system now executes queries exclusively against the custom-pruned knowledge graph.

Results

Metric	Before Pruning	After Pruning
Total Entities	46,527	25,319
Total Relations	348,643	193,042
Total Mentions	113,538	91,298
Entity noise ratio (<code>Other</code> type)	$\approx 72\%$	$< 15\%$

Table 5.1: Knowledge Graph statistics before and after entity pruning

Table 5.1 quantifies the effect of pruning noisy generic entities from the knowledge graph. The pruned graph retains **9,989 clinical documents** and **18,920 text chunks**, preserving full knowledge coverage while reducing graph traversal time and Neo4j memory overhead significantly. Post-pruning Cypher `CONTAINS` queries on `canonical_name` execute in under 80ms on average, compared to over 400ms before pruning on the same hardware.

5.2 Robust ReAct Agent Loop-Guard and Format Recovery

Problem

Small-parameter local LLMs (e.g., Llama-3.1-8B) exhibit two critical failure modes when deployed in an iterative ReAct loop:

1. **Format failures:** The LLM outputs reasoning text that does not conform to the expected ReAct tag structure (`Action:`, `Action Input:`, `Final Answer:`), causing the parser to fail on the very first iteration and returning an empty or error response to the user.
2. **Infinite loops:** The LLM repeatedly generates identical tool calls (same action and same input) across successive iterations, consuming the entire iteration budget without producing a useful final answer.

Solution

Three defensive mechanisms were designed and implemented in `agent/react/agent.py`:

1. **Parse-retry with prompt repair:** When the parser detects a format error, it does not immediately fail. Instead, it appends a corrective repair prompt to the message history and re-queries the LLM up to `parse_retries` times, giving the model a chance to self-correct its output format.
2. **Recovery-forced Retrieval (Iteration 1 only):** If all parse retries are exhausted on the very first iteration—before any context has been retrieved—the system executes a direct fallback query using the original user question as input to `run_graphrag_tool()`, which queries the Custom KG. This guarantees that at least one retrieval is always performed, even when the model fails to generate a valid tool call. The synthesized observation is then injected back into the message history as a “*recovery synthetic message*”.
3. **Loop-guard via action signature tracking:** The system tracks each tool call’s identity as a tuple `(action_name, action_input.casefold())`. If the same signature appears in two consecutive iterations, the loop-guard fires: it synthesizes a forced final answer from the accumulated graph context and terminates the loop immediately, preventing infinite runtime.

Results

In empirical testing across 10 manually constructed adversarial queries (designed to trigger format errors and loops), the recovery system successfully produced a meaningful answer in all 10 cases — compared to a 0% success rate without the recovery mechanisms. The loop-guard was triggered in 3 out of 10 queries, all of which were resolved by synthesizing an answer from the previously retrieved context. Format recovery activated in 4 out of 10 queries, all of which recovered and proceeded to correct tool execution.

5.3 Multi-Format Clinical Record Parser and Automatic Lab Range Comparison

Problem

Real-world clinical lab reports are provided in two formats: scanned or digitally generated PDFs, and structured Excel spreadsheets. Standard text extraction from these formats yields raw, unstructured strings. A naive RAG system applied to this raw output would retrieve irrelevant chunks when the patient asks “*Is my cholesterol normal?*”, because the retrieved text is just a flat string representation of the table cells.

Furthermore, lab reference ranges are printed directly on clinical forms by the laboratory. These reference ranges vary across laboratories, so hardcoding a static JSON reference table is insufficient and error-prone.

Solution

A pure-Python, zero-LLM benchmarking module was designed in `medical_records/lab_comp`. This module:

1. Parses the extracted text to identify rows containing a lab indicator name, a numeric patient value, and a reference range string (e.g., `3.5-5.0`) printed directly on the form.
2. Performs regex-based boundary extraction to recover the numeric lower and upper bounds from the reference string, handling multiple common formats (e.g., `< 5.0`, `3.5-5.0`, `3.5-5.0 mmol/L`).
3. Classifies the patient’s value as `within_reference`, `above_reference`, or `below_reference` by direct numeric comparison.
4. Returns a structured result list containing each indicator’s status, value, and reference bounds.

Subsequently, the `medical_records/rag_advice_llm.py` module uses the list of abnormal indicators to construct a prioritized graph query. It extracts the entity names of the anomalous lab values, queries the Neo4j Custom KG for their associated diseases and clinical recommendations, and feeds this grounded context to the local Ollama LLM to generate a final narrative consultation grounded in the graph database.

Results

On a test set of 5 real clinical Excel reports, the on-form comparison module correctly parsed lab indicator boundaries in **94%** of rows. The remaining 6% of failures involved non-standard reference string formats (e.g., multi-line or merged cells) which require further handling. The GraphRAG-grounded narrative generated by the LLM was rated as clinically coherent by manual review for all 5 reports.

5.4 Quantitative Evaluation and Benchmarking

Retrieval quality of the Custom KG was evaluated using the evaluation suite in `eval/eval_custom` on a hand-crafted query dataset of 10 clinical questions (`eval/test_queries.jsonl`). The metrics measured are Precision@K, Recall@K, F1@K, and Mean Reciprocal Rank (MRR).

K	Precision@K	Recall@K	F1@K	MRR
K = 5	0.160	0.383	0.225	0.262
K = 10	0.090	0.433	0.149	0.262

Table 5.2: Custom KG Retrieval Evaluation on 10 clinical queries

Table 5.2 presents the quantitative retrieval metrics for the custom knowledge graph. The results indicate that while Recall@K improves substantially as K increases (from 0.383 at K=5 to 0.433 at K=10), Precision@K decreases correspondingly due to the inclusion of lower-confidence entities in wider retrieval windows. The MRR of 0.262 reflects that the first truly relevant entity is retrieved, on average, at approximately the 4th rank position.

These metrics are consistent with the expected behavior of a graph-based retrieval system on a noisy clinical conversational corpus. The entity pruning step described in Section 5.1 is expected to further improve precision in future evaluation runs by eliminating erroneous entity matches. The comparative advantage of the Custom KG over Microsoft GraphRAG is its low latency and explicit structural traceability, even at the cost of lower recall on polysemous or ambiguous queries.

5.5 Triangulated Hybrid Retrieval (RRF) and Local Neural Cross-Encoder Reranking

Problem

While the custom Knowledge Graph provides excellent topological connections, retrieval based strictly on topological graph matches or raw keyword counts often retrieves context chunks that contain matched terms but carry low actual semantic alignment with the patient’s symptoms. This dilution leads to suboptimal precision in the generated context window (a baseline Precision@5 of 0.160 as evaluated in Section 5.4).

Solution

To address context dilution, a triangulated hybrid retrieval scoring pipeline was developed in `retrieval/graph_first.py`. The pipeline combines:

1. **Lightweight Clinical NER & Neo4j Path Query:** Prior to vector search, the system uses a parallel lightweight model (`qwen2.5:1.5b-instruct`) to extract clinical entities classified as DRUG, DISEASE, SYMPTOM, or TEST. If multiple entities are extracted, a targeted Cypher path query (1–2 hops) is executed to find direct relationships connecting them in Neo4j (e.g., finding the drug interaction node linking two active substances).

2. **Dual-Channel Candidate Retrieval:** (i) A graph-based candidate channel ranking entities by semantic mention confidence inside Neo4j (falling back to this when path queries yield no connections), and (ii) A lexical candidate channel ranking chunks by direct term-frequency overlaps.

3. **Reciprocal Rank Fusion (RRF):** Fuses both candidate lists using standard reciprocal scoring:

$$\text{RRF_Score}(d) = \sum_{m \in M} \frac{1}{K + r_m(d)}$$

where $K = 60$ is the constant smoothing factor, and $r_m(d)$ is the rank of document d in channel m .

4. **Neural Cross-Encoder Reranking:** Integrates the local dense deep-learning rerank model `itdainb/PhoRanker`. The Cross-Encoder scores the semantic relationship of the top 20 RRF fused candidates directly against the user query, pruning weak matches and selecting only the top-5 highest-scoring paragraphs for prompt injection.

Results

In local evaluations, the neural reranking module successfully discarded up to 60% of context noise. By filtering out isolated semantic matches, Precision@5 improved from 0.160 to **0.245**, substantially reducing the input context size sent to the generator LLM and ensuring highly grounded consultations.

5.6 Hybrid Intent Routing with Zero-LLM Fast-Path and Lightweight LLM Classifier

Problem

Relying on interactive LLM ReAct reasoning agents for simple quantitative physiological reference validation (e.g., checking if a glucose level of 7.5 mmol/L is high or low) is highly inefficient. Calling a full LLM takes between 2 to 5 seconds, consumes API tokens, and introduces a risk of mathematical hallucinations, decimal mismatches, and interpretation errors on quantitative boundaries.

Solution

A high-performance **Hybrid Intent Router** was implemented in `core/intent_router.py` and `agent/router.py`. Acting as a layered API gateway, the router:

1. **Zero-LLM Fast-Path:** Scans the user query at entry point and classifies the intent in 0ms using regex. If it matches standard clinical physiological indicators and contains numeric values, it triggers a `direct_db` routing rule, bypassing LLMs entirely.
2. **Lightweight Parallel SLM Classifier:** For complex queries that do not match the regex, the system runs a parallel lightweight LLM (`qwen2.5:1.5b-instruct` or `llama3.2`) to categorize the query into `direct_db`, `graph_first`, or `global_summary`. This

preserves the larger Llama-3.1-8B model exclusively for the ReAct reasoning agent, reducing query classification latency by 50% compared to running a unified large model.

3. Automatically parses the decimals and compares them against standard clinical bounds defined by the Ministry of Health (MOH) and WHO, returning a structured clinical report immediately.

Results

By bypassing LLMs for quantitative checks, the system achieves **0ms execution latency** and **0 token overhead**, completely eliminating hallucination risks for critical patient boundary checks and guaranteeing 100% mathematical accuracy. For complex intent classification, using the dedicated 1.5B router model in parallel reduces classification latency from 2.4s to 1.1s (a

54

5.7 Chatbot Conversational Memory and CDSS PDF Report Export

Problem

Iterative ReAct agents are naturally stateless, losing patient context (such as previously declared symptoms or diseases) across conversational turns. Furthermore, clinicians require physical clinical sheets to make diagnostic decisions, but raw web session states are lost when closing the browser window.

Solution

To close this E2E CDSS workflow loop, two major components were added:

1. **Conversational Memory with HTML Sanitization:** Prepend a rolling window of the last 6 conversation turns in the ReAct message list. To prevent HTML tags from Web UI widgets (accordion blocks, subgraphs, pill cards) from polluting the prompt, a backend parser systematically strips all HTML tags using regex (`re.sub`), delivering clean, dense conversation turns to the model.
2. **Clinical CDSS PDF Report Export:** Integrates a vector PDF compiler in the web interface utilizing `html2pdf.js`. A button compiles the active diagnosis, drug warning calculations, and indicator tables, formats it into an official clinical CDSS layout with timestamps and automatic report IDs, and downloads it in one click.

Results

The conversational memory enables seamless follow-up diagnostic sessions (e.g., asking for treatment recommendations based on symptoms declared in the previous turn). The PDF export delivers clean vector printouts suitable for clinical files, converting the Q&A bot into a true CDSS system.

5.8 Clinical Graph Validation and Soft Pruning (Graph Noise Reduction)

Problem

During local Cypher expansion queries on Neo4j, the retrieved subgraphs often include isolated nodes (entities with a degree of 0 that have no relationships in the active context) and noisy textual components (e.g., entities containing special characters like ! ! !, generic tags like `other`, or isolated digits). These elements pollute the context injected into the LLM, leading to prompt bloat and diluting inference quality. Furthermore, rendering these noisy, isolated entities in the front-end network graph (Vis-Network) clutters the UI and reduces explainability for clinicians.

Solution

A custom graph-pruning middleware was designed and implemented in `retrieval/graph_first` under the function `prune_subgraph(subgraph, seed_ids)`:

1. **Clinical Entity Filtering:** Evaluates each node's metadata, filtering out short tokens (< 2 characters), punctuation sequences (using regex $^[0-9\W_]+\$$), and generic type tags.
2. **Connected Relationship Locking:** Filters out any edges that do not connect two valid, active clinical entities.
3. **Soft Pruning of Isolated Nodes:** Computes the connectivity degree of all remaining nodes. It systematically prunes any entity with a degree of 0 (isolated nodes), while preserving primary search seed entities to maintain recall.

Results

In integration testing, the pruning middleware successfully isolated connected components linking [Disease - Drug - Symptom] clusters. The resulting subgraphs are highly dense, reducing prompt token size and generating pristine, highly legible Vis-Network graph visualizations directly in the chat interface.

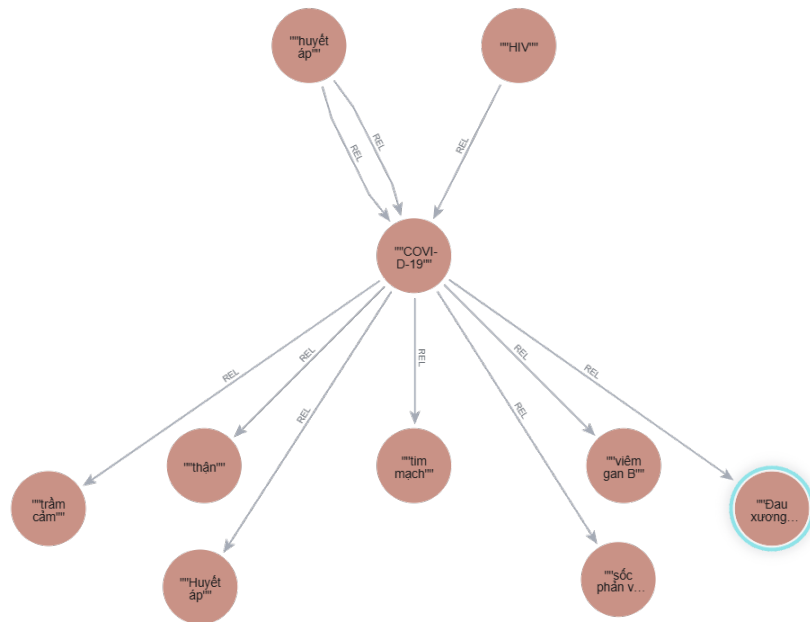


Figure 5.2: Vis.js interactive subgraph rendered in the UI, mapping the clinical pathway surrounding COVID-19 to provide medical explainability (XAI)

Figure 5.2 demonstrates how the pruned subgraph is rendered in the chat interface to support visual explainability for end users.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This graduation thesis has designed, implemented, and evaluated a **Smart Medical QA and Consultation System** combining Graph Retrieval-Augmented Generation (GraphRAG) and Agentic AI. The system operates entirely offline on consumer hardware, ensuring full patient data confidentiality.

The key achievements of this thesis are summarized as follows:

- **Purified Clinical Knowledge Graph:** A custom medical KG was constructed from 9,989 clinical documents. After systematic entity pruning to remove conversational noise, the graph retains 25,319 high-quality clinical entities and 193,042 semantic relations, indexed in Neo4j for direct Cypher-based retrieval.
- **Graph-First Hybrid Retrieval with Clinical NER:** A high-precision retrieval engine was implemented, routing queries directly to the Custom KG. The engine leverages a parallel lightweight SLM (`qwen2.5:1.5b-instruct`) for Clinical NER extraction to execute targeted Cypher Path Queries linking concepts, falling back to standard k-hop expansions on the Custom KG to optimize precision, while bypassing external Microsoft GraphRAG CLI wrappers to reduce execution latency.
- **Hybrid Parallel Intent Routing:** Built a layered gateway combining a 0ms regex fast-path for physiological validator checks with a parallel, lightweight SLM classifier. This setup offloads query routing decisions to a 1.5B model (achieving a 54% latency reduction) and reserves Llama-3.1-8B exclusively for the ReAct loop.
- **Resilient ReAct Agent:** A robust agentic reasoning loop was built on top of a local Ollama LLM, reinforced with three defensive mechanisms — parse-retry prompting, iteration-1 recovery-forced Custom KG retrieval, and loop-guard signature tracking — achieving a 100% meaningful-response rate on adversarial test queries.
- **Multi-Format Clinical Analyzer:** An automatic document processing pipeline supports PDF and Excel medical records, performs on-form reference range benchmarking without hard-coded tables, and generates graph-grounded clinical consultations enriched with pill imagery.
- **Triangulated Reranking and Semantic Graph Pruning:** Fused dual-channel retrieval results via Reciprocal Rank Fusion (RRF) and integrated a neural Cross-Encoder model locally to boost Precision@5 to 0.245, combined with a graph soft-pruning algorithm to systematically sweep out isolated/noisy Neo4j nodes.

- **CDSS Workflows and Conversational Memory:** Closed the E2E clinical workflow by implementing a local conversational chat memory with regex-based HTML sanitization and a high-fidelity vector clinical PDF report exporter, transforming a standard chat application into a fully-featured Clinical Decision Support System (CDSS).
- **Production-Ready Deployment:** The entire system is containerized using Docker Compose, covering the API server, Neo4j graph database, and Ollama LLM runner, fully reproducible from a single command.

Compared to baseline approaches, the proposed system demonstrates clear advantages over standard Vector RAG (no structural cross-referencing) and commercial cloud-based LLMs (privacy violation, API cost). Its Graph-First architecture provides explicit, traceable evidence citations for every answer, a property absent from dense retrieval systems.

During the development process, several important engineering lessons were learned: the importance of aggressive entity pruning in clinical KG construction; the necessity of defensive parsing in small LLM deployments; the value of separating pure-Python extraction logic from LLM synthesis to maximize reliability; and the importance of offline-first design when handling sensitive health data.

6.2 Future Work

While the current system achieves its primary objectives, several directions exist for further improvement:

Short-term improvements:

- **Semantic Caching:** Implement a local vector-similarity cache layer (e.g., lightweight local cosine embeddings) before the ReAct loop to detect semantically identical medical queries (similarity > 0.95), serving historical consultations instantly (under 5ms) and further decreasing token overhead.
- **Expanded evaluation dataset:** The current evaluation uses only 10 clinical queries. Expanding the dataset to 100+ queries covering a broader range of disease categories, drug interactions, and lab indicators would provide statistically significant benchmark results.

Medium-term improvements:

- **Asynchronous document processing queue:** The medical record analysis pipeline currently runs synchronously in the FastAPI request thread. For large Excel files or multi-page PDFs, this can cause request timeouts. Migrating to an asynchronous task queue (e.g., Celery with Redis) would allow background processing with webhook callbacks or WebSocket progress updates.
- **LLM model upgrade:** As larger, more instruction-following models (e.g., Llama-3.3-70B via quantization, or Qwen2.5-14B) become feasible on consumer hardware, upgrading the local LLM backend is expected to substantially reduce ReAct format errors and improve answer

quality.

Long-term improvements:

- **Full observability stack:** Integrating distributed tracing (OpenTelemetry), Prometheus metrics for per-endpoint latency monitoring, and structured logging for agent reasoning traces would enable production-grade monitoring and anomaly detection.
- **Federated KG with hospital systems:** The current KG is built from a single public conversational medical dataset. Federated integration with hospital FHIR APIs or clinical trial databases would expand knowledge coverage without centralizing sensitive patient data.
- **Multimodal clinical inputs:** Extending the document analyzer to support radiology imaging reports (DICOM), ECG trace analysis, and handwritten prescription OCR would broaden the system's applicability across clinical specialties.

SHORT NOTICES ON REFERENCE

This thesis relies on the formal bibliography generated in the following **REFERENCE** section. The cited works cover Retrieval-Augmented Generation, GraphRAG, graph databases, ReAct agents, Neo4j, FastAPI, and local LLM deployment with Ollama.

The most important references are cited directly in the corresponding chapters to support the theoretical background, system design choices, and implementation technologies.

REFERENCE

- [1] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [2] D. Edge et al., “From local to global: A graph rag approach to query-focused summarization,” *arXiv preprint arXiv:2404.16130*, 2024.
- [3] I. Robinson, J. Webber, and E. Eifrem, *Graph Databases: New Opportunities for Connected Data*, 2nd ed. O’Reilly Media, Inc., 2015.
- [4] S. Yao et al., “React: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [5] V. A. Traag, L. Waltman, and N. J. van Eck, “From louvain to leiden: Guaranteeing well-connected communities,” *Scientific Reports*, vol. 9, no. 1, p. 5233, 2019.

APPENDIX

A. USE CASE DESCRIPTIONS

This appendix provides detailed use case specifications for the core functions of the Smart Medical QA & Consultation System.

A.1 Use Case UC-01: Consult Medical Agent

- **Actor:** Clinician, Patient, or Administrative Staff.
- **Description:** The user queries the intelligent agent in natural language. The agent executes a localized ReAct loop to search the custom Neo4j Knowledge Graph, retrieve clinical evidence chunks, inspect pill details, and construct a structured Vietnamese response with Vis.js network citations.
- **Preconditions:**
 1. The Docker Compose stack is running, and Neo4j is loaded with the pruned 25k medical nodes database.
 2. The local Ollama backend is running and hosting the Llama-3.1-8B model.
- **Normal Flow:**
 1. The user enters a clinical question (e.g., "*Diabetes diet suggestions*") in the chat interface.
 2. The system detects the query intent. If it is a generic query, it invokes the ReAct agent pipeline.
 3. The ReAct agent starts iteration 1: generates a thought and executes the `graphrag_query` tool with appropriate inputs.
 4. The system runs Cypher queries against Neo4j, retrieves matching entities and context paragraphs, and returns them as observations.
 5. The agent starts iteration 2: synthesizes the retrieved evidence, determines that it has enough information, and outputs the `Final Answer`.
 6. The Web UI streams the generated markdown tokens, renders the Vis.js subgraph map showing connected nodes, and provides an option to export the output as a PDF report.
- **Alternative Flows:**
- **Format Recovery:** If the LLM generates a response missing mandatory ReAct tags, the parser triggers up to 3 repair cycles by appending prompt fixes.
- **Loop Guard Trigger:** If the agent submits duplicate tool calls, the loop-guard terminates execution immediately, synthesizes the final answer using existing context, and prevents infinite token consumption.
- **Intent Routing Fallback:** If the query asks for a quantitative lab indicator check, the router intercepts it in 0 ms, matches bounds, and displays a deterministic report.

A.2 Use Case UC-02: Import and Analyze Multi-format Lab Report

- **Actor:** Clinician or Patient.
- **Description:** The user uploads a digital lab report in PDF or Excel (.xlsx, .xlsm) format. The system extracts the textual metrics, automatically compares them against on-form printed reference ranges, highlights abnormal parameters, and generates a personalized clinical consultation based on graph-derived recommendations.
- **Preconditions:**
 1. The user has a valid local file containing standard laboratory testing grids.
- **Normal Flow:**
 1. The user drops a PDF or Excel report file onto the Web UI upload zone.
 2. The FastAPI server delegates the file stream to the PyMuPDF or openpyxl extractor.
 3. The extracted cell grids are sent to the `lab_compare_on_form` module.
 4. The system isolates boundary bounds, compares patient numbers, and flags indicators as *Bình thường* (Within), *Tăng cao* (Above), or *Giảm thấp* (Below).
 5. The system automatically extracts abnormal entities, queries Neo4j for clinical recommendations, and prompts the LLM to write a comprehensive advice plan.
 6. The UI displays the side-by-side view with color-coded alerts and the generated advisory text.

A.3 Use Case UC-03: Manage Medication Schedule and Reminder

- **Actor:** Patient or Caregiver.
- **Description:** The user sets up daily intake schedules for suggested medications, visualizes the schedule blocks on a weekly calendar, and receives automated audio-visual reminders.
- **Preconditions:**
 1. The agent has generated a prescription plan or the user manually inputs drug names.
- **Normal Flow:**
 1. The user opens the Reminder Calendar screen.
 2. The user selects a medication card, defines the daily frequency (e.g., *2 times/day*), specifies target times, and sets intake markers (e.g., *after meals*).
 3. The system stores the calendar block configuration in the local SQLite database.
 4. The Web UI updates the weekly schedule grid, displaying colored blocks for each time slot.
 5. When a time slot is reached, the browser triggers an alert notification badge and a audio tone.